

SSL/TLS Certificate Setup and Management in DevOps

1. Introduction

- 1.1 What is SSL/TLS?

SSL/TLS = Secure connection between browser & server.

TLS is the modern and secure version of SSL.

Used for https:// sites (the "s" stands for secure).

Example: When you visit https://example.com, TLS encrypts your data (like login details) so no one can steal it in between.

- 1.2 Why SSL/TLS is Important in DevOps?

Protects sensitive data during deployment.

Avoids data leaks and MITM (Man-in-the-Middle) attacks.

Ensures your app meets compliance (GDPR, PCI-DSS, HIPAA).

- 1.3 How to Manage Certificates?

- Choose based on scale:

- Let's Encrypt: Free, auto-renewal
- AWS ACM, Azure: Auto-renewal, easy to use
- Self-Signed: For internal apps/testing
- Commercial CA: For banks, enterprise apps

2. SSL/TLS Basics

- 2.1 How SSL/TLS Works (5 Steps)

1. Client sends 'hello' → lists supported encryption.
2. Server replies with certificate & selected method.
3. Client checks if certificate is trusted.
4. Keys are exchanged securely.
5. All data is now encrypted.

- 2.2 Types of Certificates

- DV: Domain only
- OV: Verifies company
- EV: Highest trust
- Wildcard: *.example.com
- SAN: example.com, api.test.com

- 2.3 What is a Certificate Authority (CA)?

Trusted company that issues SSL certificates.

Examples: DigiCert

- 2.4 Public vs Private Certificates

Public = for websites (trusted by browsers)

Private = for internal use

3. Generating Certificates

- 3.2 Let's Encrypt + Certbot

```
sudo apt install certbot python3-certbot-nginx
```

```
sudo certbot --nginx -d example.com -d www.example.com
```

- 3.3 Self-Signed Cert (Internal Testing)

```
openssl req -newkey rsa:2048 -nodes -keyout mykey.pem -out mycsr.csr
```

```
openssl x509 -req -days 365 -in mycsr.csr -signkey mykey.pem -out mycert.pem
```

- 3.4 Cloud Services

- AWS ACM: `aws acm request-certificate --domain-name example.com`

- Azure: `az keyvault certificate import`

- GCP: Use ManagedCertificate in Ingress YAML

4. Deploying SSL in Web Apps

- 4.1 NGINX Example

```
server {  
    listen 443 ssl;  
    server_name example.com;  
    ssl_certificate /etc/ssl/certs/fullchain.pem;  
    ssl_certificate_key /etc/ssl/private/privkey.pem;  
}
```

- 4.2 Cloud Load Balancers

- AWS: Add cert in ALB via ACM

- Azure: Upload .pfx to Key Vault → assign to App Gateway

- GCP: Use Ingress + ManagedCertificate

- 4.3 API Security

```
server {  
    listen 80;
```

```
return 301 https://$host$request_uri;  
}
```

- 4.4 Kubernetes Ingress Example

tls:

- hosts:
 - myapp.example.com
- secretName: myapp-tls

5. Automating Certificate Management

- 5.1 Auto-Renew with Certbot

```
sudo certbot renew --dry-run
```

- 5.2 ACME Clients

- Certbot, acme.sh, Caddy, Lego

- 5.3 Kubernetes + cert-manager

```
helm install cert-manager jetstack/cert-manager
```

6. Best Practices

- Use Strong Encryption

```
ssl_protocols TLSv1.2 TLSv1.3;
```

- Use HSTS

```
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains";
```

- OCSP Stapling

- Speeds up certificate verification

- Store Keys Securely

- chmod 600, AWS KMS, Azure Key Vault

7. Troubleshooting

- Common Fixes

- ERR_CERT_DATE_INVALID: Renew cert

- NET::ERR_CERT_COMMON_NAME_INVALID: Domain mismatch

- SSL Handshake Failed: Cipher mismatch

- Check SSL

openssl s_client -connect example.com:443

- Test Expiry

openssl x509 -noout -enddate -in cert.pem

8. Tools & Links

- <https://letsencrypt.org/>
- <https://www.ssllabs.com/ssltest/>
- <https://ssl-config.mozilla.org/>
- <https://nmap.org/nsedoc/scripts/ssl-enum-ciphers.html>